Solution: Perfect Square

Problem Statement

Write a Recursive Solution to Check if a Given Number is a Perfect Square or Not.

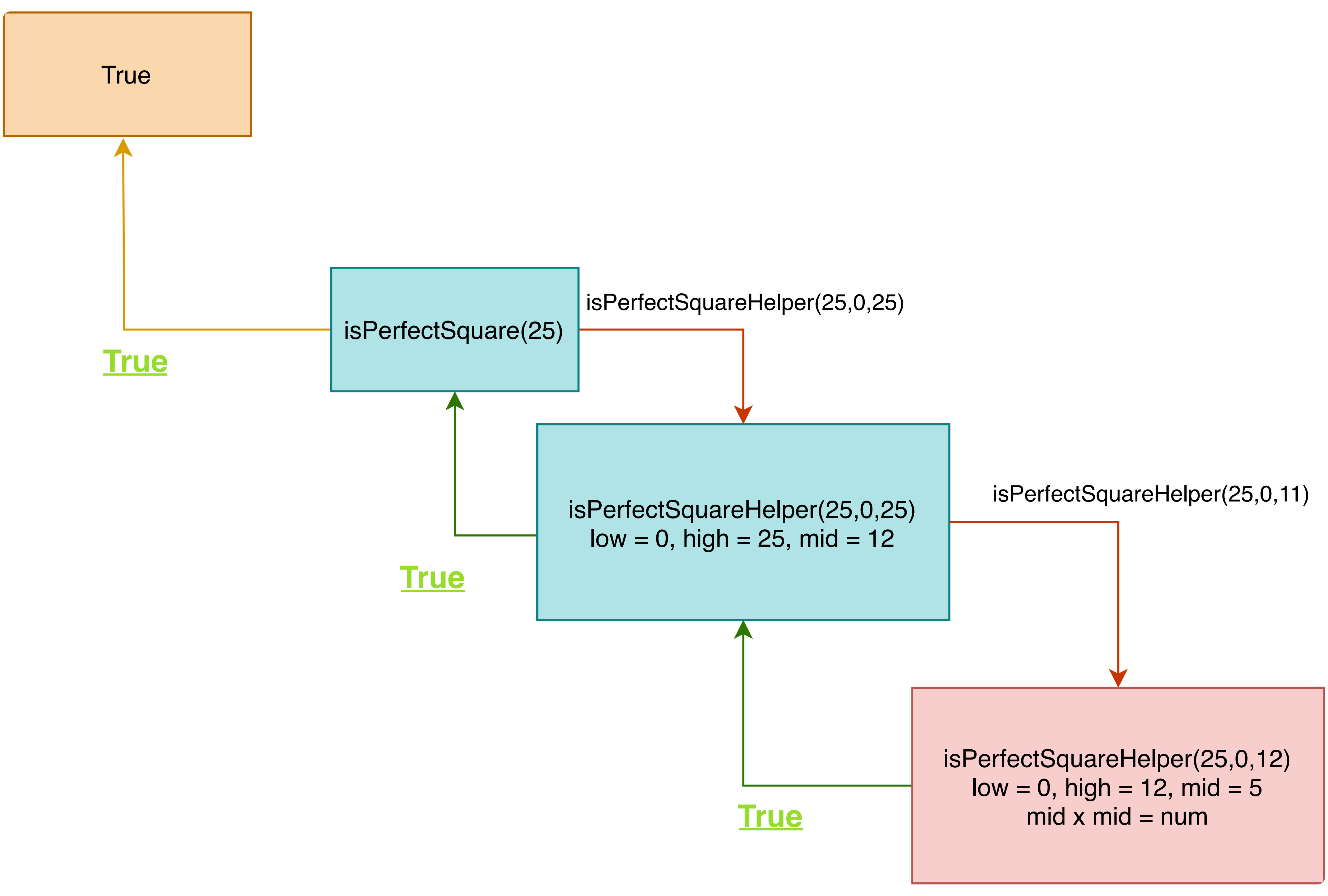
The problem is to determine whether a given positive number is a perfect square or not. A square number or perfect square is an integer that is the square of an integer; in other words, it is the product of some integer with itself.

Examples

Input	Output	Explanation
16	True	16 is a perfect square because 4 * 4 = 16.
14	False	14 is not a perfect square because there is no integer whose square is equal to 14.
9	True	9 is a perfect square because 3 * 3 = 9.

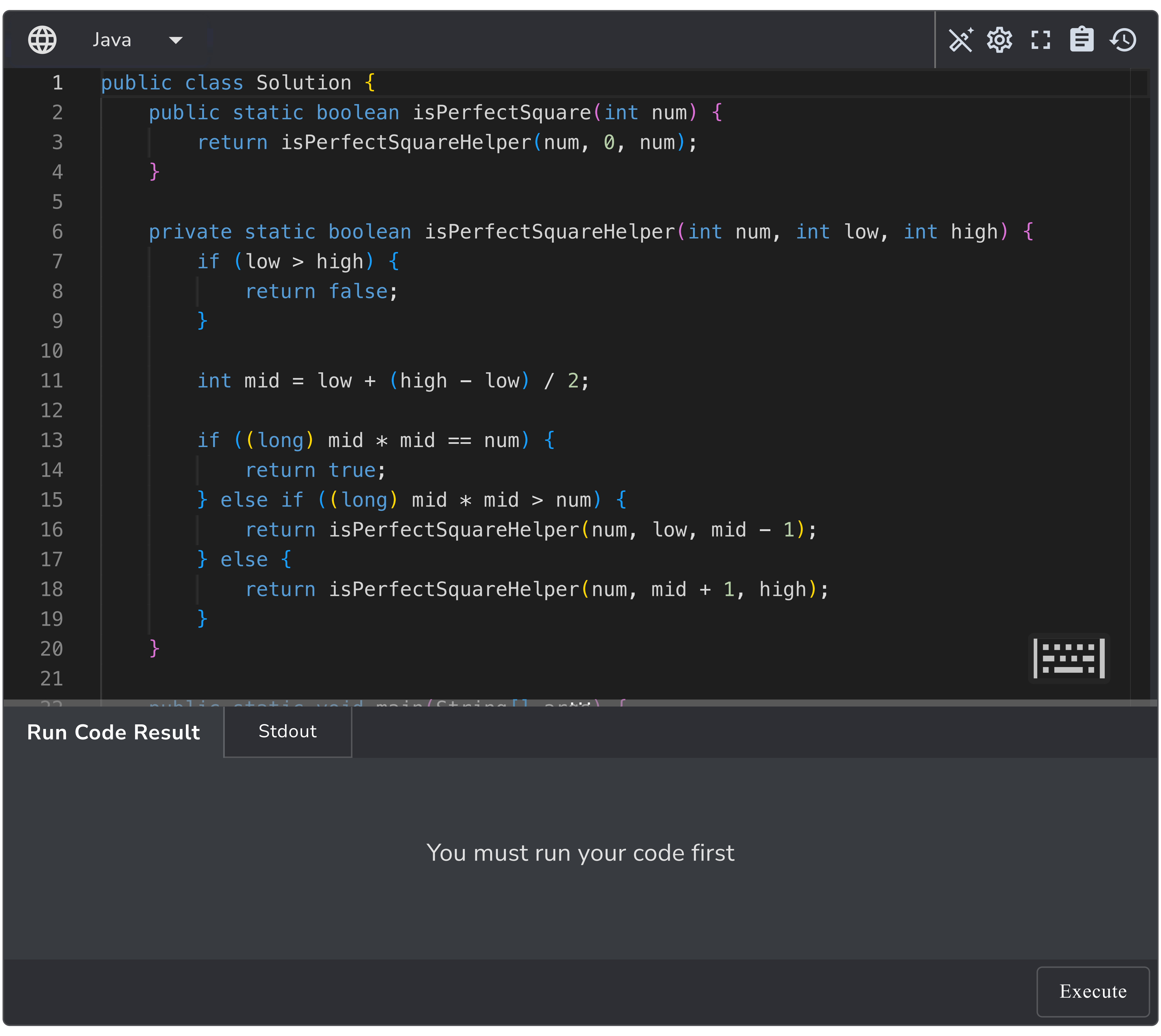
Solution

1. Base Case:
    - If the given number is 0 or 1, return true because 0 and 1 are perfect squares.
2. Recursive Case:
    - The algorithm performs a binary search to find the square root of the number and check if it is a perfect square.
    - Calculate the mid value between a lower bound and an upper bound.
    - Check if the square of the mid value is equal to the given number.
        - If it is equal, return true because the number is a perfect square.
        - If it is less than the given number, make a recursive call with an updated lower bound of mid + 1.
        - If it is greater than the given number, make a recursive call with an updated upper bound of mid - 1.
    - Repeat the process until a perfect square is found or the lower bound becomes greater than the upper bound.


True
True
isPerfectSquare(25)
isPerfectSquareHelper(25,0,25)
True
isPerfectSquareHelper(25,0,25)
low = 0, high = 25, mid = 12
isPerfectSquareHelper(25,0,11)
True
isPerfectSquareHelper(25,0,12)
low = 0, high = 12, mid = 5
mid x mid = num

Code

Here is the code for this algorithm:


Java
Run Code Result
Stdout
Execute

```java
public class Solution {
    public static boolean isPerfectSquare(int num) {
        return isPerfectSquareHelper(num, 0, num);
    }

    private static boolean isPerfectSquareHelper(int num, int low, int high) {
        if (low > high) {
            return false;
        }

        int mid = low + (high - low) / 2;

        if ((long) mid * mid == num) {
            return true;
        } else if ((long) mid * mid > num) {
            return isPerfectSquareHelper(num, low, mid - 1);
        } else {
            return isPerfectSquareHelper(num, mid + 1, high);
        }
    }
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(logN)$ because we perform a binary search on the possible square roots.

The space complexity of the algorithm is $O(logN)$ due to the recursion stack used in the binary search process.

← Previous
Next →
6. Perfect Square
7. Number Frequency
☐ Mark as Completed